

Automatic contribution bounding for grouped DP queries

1 Motivation

1.1 The gap between a DP library and a DP system

Differentially private aggregation is solved as an *algorithm* and unsolved as a *product*. Every mature library — Google DP, OpenDP, Tumult, PipelineDP — can answer `SELECT g, SUM(v) FROM T GROUP BY g` with a rigorous user-level guarantee. But none of them can answer it *without a human first supplying constants that depend on the data*:

- **How much can one user contribute to one group?** A clipping bound. Set it too low and you destroy the signal; too high and you drown it in noise.
- **How many groups can one user touch?** `max_partitions_contributed`, called C_u here. A required input in Google DP, with no default. Wilson et al. [9] do provide a mechanism for inferring bounds automatically (their APPROXBOUNDS, §5.1.1) — but it infers the *value* range $[L, U]$, and their aggregate signatures are correspondingly `ANON_SUM(col, L, U)`. C_u appears only as “an engine-defined constant” in their sensitivity theorem, with the paper stating: “*For the rest of this paper, we assume that contribution bounds are specified as literals in each query.*”

Our system rewrites ordinary SQL into a private plan automatically. An analyst writes a query; they do not know — and should not need to know — the maximum number of months any customer appears in. So every one of these constants is a blocker, and the question driving this work is narrow and practical: **can C_u be chosen from the data itself, privately, without spending so much budget that the answer becomes useless?**

1.2 Guessing is not a safe fallback

Two measured facts rule it out. The penalty is *severely asymmetric* — setting C_u too small costs up to 38× in error while setting it too large costs at most 1.7× — and no single value works across datasets:

analyst supplies	$C_u=1$	$C_u=10$	$C_u=100$	$C_u = \text{schema max}$	automatic
worst case over queries	177.6×	136.3×	5.18×	4.18×	1.36×

$C_u=1$ is catastrophic on TPC-H and simultaneously *optimal* on ClickBench. A single constant cannot serve both, so the choice must adapt to the data — which means spending privacy budget to make it. The mechanism below spends about 1% of ϵ doing so.

1.3 Where the budget goes

A grouped DP query spends privacy budget on three things:

1. **Bound selection** (ϵ_b): how large is one user’s contribution?
2. **Partition selection** (ϵ_η): which groups may be named at all? A group existing because of a single user must not be revealed, so groups are released only when a noised count of contributing users clears a threshold τ .
3. **Values** (ϵ_v): the noised aggregates themselves.

The third is what the analyst wants; the first two are overhead.

How the baseline divides it. Google DP charges these separately, though not as three co-equal shares. Privacy on Beam splits the top level *two* ways — `count.go` exposes `AggregationEpsilon` and `PartitionSelectionEpsilon` as distinct fields, consumed by distinct calls — and bound selection sits *inside* the aggregation: `ApproxBounds` is constructed with its own `epsilon` taken from the aggregation’s share. We model all three as a single three-way split and tune it jointly for every arm, which is at least as favourable to the baseline as its own nested structure, since a nested split cannot beat a free one.

Scoring. Throughout: relative ℓ_1 error over the *true, uncapped* key set, with τ -suppressed groups charged as released 0 (full error). Every arm is tuned separately over its own free parameters. Anything less lets a mechanism buy accuracy by returning fewer rows, or flatters one side by fixing a parameter the other gets to optimise.

2 The mechanism

Core idea. The bound we need is a property of one *distribution* — how many groups each user touches. Rather than measuring that distribution privately and reading a bound off it, we turn the problem into a *choice among candidate bounds*, scored by a quantity that a single user can move by at most one. Choosing is far cheaper than measuring, and the asymmetry of the error curve tells us which way to lean when the choice is imperfect.

2.1 In words

Ask each user how many groups they appear in. On a monthly sales query one customer might appear in 30 months, another in 3, a few in 70. Lay those counts out as a distribution and C_u is a cut point on it: choose it, and every user above the line has their extra groups discarded.

The mechanism sets that cut point at a *quantile* — “pick C_u so that 95% of users are left untouched” — and it does so privately, because the distribution is itself sensitive data. Three design choices make this work.

Aim at a rank, not a value. Rather than asking “what is the largest k_u ?”, which one user can move arbitrarily, we ask “at what cut point do we reach 95% of users?”. That target is a *count* of users, and one user changes any count by at most one. The bound is thus pinned to a robust feature of the distribution, not to its most extreme member.

Choose, do not measure. The obvious approach is to release the whole distribution noisily and then read the quantile off it. That is expensive: one user with $k_u = 1$ shifts every cumulative count at once, so publishing the full curve requires noise proportional to its length. Instead we never publish the curve. We *score* each candidate cut point by how close it lands to the target rank, and draw one candidate at random with better candidates exponentially more likely. Only the winner is released. Since each individual candidate’s score moves by at most one when a user is added or removed, the whole selection costs the same as measuring a single count — about 1% of the query’s budget in practice.

Deliberately overshoot. A quantile is only a proxy for the bound that actually minimises error, so the two will not coincide. That is tolerable because the error curve is lopsided: under-shooting discards real user data and costs up to $35\times$, while overshooting merely adds noise and costs at most $2\times$ (§3.2). The target fraction p is therefore set high — 0.95 or 0.99 rather than the 0.7 one might first reach for — so the mechanism errs on the safe side of a curve that punishes one direction far more than the other.

Because the resulting bound depends only on the shape of the k_u distribution, and that shape is stable under filters on the user (§3.4), the released value can be frozen and reused across many later queries at no further cost.

2.2 Formally

Let $k_u = |\{g : u \text{ contributes to } g\}|$ and $F_d(r) = |\{u : k_u \leq r\}|$, the number of users whose group count is at most r . Given a public upper bound $K \geq k_u$ and a target fraction p :

1. Release $\tilde{N} = N + \text{Lap}(1/\epsilon_N)$, the number of users, at cost $(\epsilon_N, 0)$. Its sensitivity is 1.
2. Form a conservative target rank $T = p\tilde{N} + (p/\epsilon_N) \ln(1/2\beta_N)$; the margin ensures $T \geq pN$ with probability $1 - \beta_N$, so noise in $\tilde{N}$ does not silently pull the target below its intended quantile.
3. Score each candidate $r \in \{1, \dots, K\}$ by $q(d, r) = -|F_d(r) - T|$: a candidate scores well when the number of users it leaves untruncated is close to the target.
4. Sample by exponential mechanism, $\Pr[\hat{C}_u = r] \propto \exp(\epsilon_C q(d, r)/2)$, at cost $(\epsilon_C, 0)$. The histogram, the cumulative counts and the scores all remain internal; only $\hat{C}_u$ is released.

Total $(\epsilon_N + \epsilon_C, 0)$ -DP by adaptive sequential composition. $\hat{C}_u$ is then frozen and reused as public metadata.

2.3 What we changed

The construction above is Dandan’s; steps 1–4 and the $\Delta q \leq 1$ argument are hers. Our contribution to the mechanism itself is three modifications, each forced by a measurement.

The target fraction, $0.7 \rightarrow 0.95/0.99$. The original proposal suggested $p = 0.7$ as a reasonable default. Measured across nine queries, that is the single worst choice tested: it costs $17.75\times$ in the worst case, against $1.36\times$ at $p = 0.99$. The reason is the asymmetry of §3.2 — at $p = 0.7$ the selected bound lands *below* the error-optimal one, on the steep side of the curve. The instinct to err high was right; the data says to go much further.

The right p depends on the surrounding mechanism. A single value does not serve both architectures. Where C_u is shared with the partition-selection threshold — the published baseline’s structure — a large C_u inflates τ as well, and chasing a heavy tail backfires: $p = 0.99$ costs $2.46\times$ against $2.31\times$ at $p = 0.95$. Where the value and vote bounds are separate, $p = 0.99$ wins outright ($1.36\times$ against $2.91\times$). This interaction was not part of the original proposal and changes the recommendation depending on where the mechanism is deployed.

When to re-select rather than reuse. The proposal freezes $\hat{C}_u$ and reuses it. We find that is correct for filters on the privacy unit — where the k_u distribution is *identical* even after discarding 99% of users — but wastes $1.8\times$ on filters that remove whole groups, where k_u collapses and re-selection pays for itself. The decidable test is in §3.4, and it is not simply “does the WHERE clause touch the grouping key”.

Two safety modifications were needed for the mechanisms this one feeds (§4): a clip written as a signed sum rather than a norm, and a δ that did not cover every group a user can create. Neither is in the selection mechanism itself, but both must be fixed in anything that consumes its output.

2.4 Why the score sensitivity is 1, not K

This is what makes step 4 cheap. Removing one user with $k_u = k$ shifts $K - k + 1$ cumulative counts *simultaneously*, so *releasing* the vector $(F(1), \dots, F(K))$ would have ℓ_1 sensitivity up to K . But the exponential mechanism never releases that vector — it scores each candidate separately, and what matters is $\Delta q = \max_r |q(d, r) - q(d', r)|$. Verified by explicit construction:

removed unit	cumulative counts changed	ℓ_1 change	$\max_r \Delta q $
$k_u = 1$	80 of 80	80	1.0
$k_u = 30$	51 of 80	51	1.0
$k_u = 70$	11 of 80	11	1.0

The ℓ_1 of the histogram is a red herring: one cheap release replaces K noisy counts.

3 Results

3.1 The selection is nearly free and essentially deterministic

With $\sim 180k$ units, F jumps by thousands between adjacent candidates, so the exponential mechanism’s distribution is sharply peaked. All 15 draws agree at every budget tested, and *spending more actively hurts*, because it comes out of the query:

$\epsilon_N + \epsilon_C$	0.010	0.025	0.100	0.250
selected C_u , 15 draws	37–37	37–37	37–37	37–37
penalty vs oracle C_u	1.00 $\times$	1.00 $\times$	1.04 $\times$	1.16 $\times$

About 1% of ϵ is the right budget.

3.2 The target fraction matters, and $p = 0.7$ is too low

Penalty is error at the selected C_u over error at that query’s best C_u , so 1.00 $\times$ equals an oracle that already knew the answer:

query	best C_u	$p=0.7$	$p=0.9$	$p=0.95$	$p=0.99$
tpch month	55	17.75 $\times$	2.91 $\times$	2.91 $\times$	1.35 $\times$
tpch month nation	44	1.18 $\times$	1.02 $\times$	1.00 $\times$	1.21 $\times$
tpch month prio	55	1.41 $\times$	1.03 $\times$	1.00 $\times$	1.17 $\times$
so posts month	44	2.02 $\times$	1.56 $\times$	1.31 $\times$	1.22 $\times$
so comments month	34	1.70 $\times$	1.34 $\times$	1.11 $\times$	1.36 $\times$
cb hits region	1	1.02 $\times$	1.00 $\times$	0.99 $\times$	1.00 $\times$
worst case		17.75 $\times$	2.91 $\times$	2.91 $\times$	1.36 $\times$

The reason is the shape of the error curve around the optimum, which is severely asymmetric:

query	$C_u/8$	$C_u/4$	$C_u/2$	C_u	$2C_u$	$4C_u$
tpch month	137.7 $\times$	104.0 $\times$	35.3 $\times$	1.00 $\times$	1.94 $\times$	2.49 $\times$
tpch month nation	7.32 $\times$	5.93 $\times$	3.14 $\times$	1.00 $\times$	1.96 $\times$	3.11 $\times$
tpch month prio	9.11 $\times$	6.62 $\times$	2.60 $\times$	1.00 $\times$	1.70 $\times$	1.70 $\times$
so posts month	1.44 $\times$	1.32 $\times$	1.13 $\times$	1.00 $\times$	1.28 $\times$	1.67 $\times$

Halving C_u costs 2.6–35 $\times$; doubling costs 1.7–2.0 $\times$. **The selection should sit deliberately above the optimum.**

What p means concretely. p is the fraction of *users* who escape truncation, which is not the same as the fraction of groups they may occupy:

query	groups	$p=0.7$	$p=0.9$	$p=0.95$	$p=0.99$
tpch month	84	37 (44%)	45 (54%)	48 (57%)	63 (75%)
tpch month nation	2,095	37 (1.8%)	45 (2.1%)	48 (2.3%)	62 (3.0%)
so posts month	190	1 (0.5%)	3 (1.6%)	8 (4.2%)	120 (63%)
users truncated		24–38%	7–11%	2–11%	0.0–0.4%

At $p = 0.99$ on a fine grouping this is modest (3% of groups); on a coarse one it does approach “may appear almost everywhere”, which is correct there, because with few groups truncation bias dominates and noise is comparatively cheap.

The right p depends on the surrounding mechanism. If C_u is shared with τ — the published baseline’s structure — then a large C_u inflates the partition-selection threshold too, and chasing a heavy tail backfires:

worst case over queries	$p=0.7$	$p=0.9$	$p=0.95$	$p=0.99$
C_u shared with τ (published)	15.60×	2.50×	2.31×	2.46×
C_e, C_v separate	17.75×	2.91×	2.91×	1.36×

Recommendation: $p \approx 0.95$ with a shared C_u , $p \approx 0.99$ with separate bounds. Not 0.7.

3.3 Multiple aggregates

Real queries request several measures at once, which splits ϵ_v c ways while the partition-selection cost is paid once. Measured on `month|nation` with up to four aggregates (SUM price, SUM quantity, COUNT, SUM discount):

aggregates c	1	2	3	4
baseline	13.52%	18.93%	22.86%	26.25%
ℓ_1 clip	3.89%	8.02%	11.40%	13.40%
advantage	3.48×	2.36×	2.01×	1.96×

The clip’s advantage decays and appears to flatten near 2×: τ is a fixed cost paid once regardless of c , so as c grows both arms converge toward pure value-channel noise where the gap is just the sensitivity ratio. **Single-aggregate figures should therefore be read as the upper end of a range.**

The selection mechanism is unaffected: C_u is chosen once per query, so its $\epsilon_N + \epsilon_C$ is paid once regardless of c and its relative cost stays at $\approx 1\%$ of the query budget. (Reasoned, not measured — the experiment above varies the value channel, not the selection.)

3.4 Freeze once, or re-select per query?

A single frozen C_u suffices for filters on the *privacy unit*; only group-removing filters justify re-selection. Frozen $C_u = 48$ from the unfiltered query; per-query re-runs the mechanism and pays 0.01 again each time:

query	units	k_u p50	p95	per-query C_u	frozen	per-query
PU-side: acctbal ≥ 8000	181,532	30	48	48	12.36%	12.56%
PU-side: acctbal ≥ 9500	45,320	30	48	53	85.14%	87.16%
PU-side: acctbal ≥ 9900	9,182	30	48	57	100.0%	99.99%
GRP-side: ship ≥ 1998	821,108	4	8	8	2.52%	1.38%
GRP-side: nation < 5	199,738	30	48	48	2.31%	2.40%

A filter on the privacy unit removes users, but the survivors still touch the same number of groups — the k_u distribution is *identical* (p50 = 30, p95 = 48) even at a filter retaining 9,182 of 1,000,000 users. Re-selection returns the same value and is slightly worse for paying its cost again. A filter that removes *groups* collapses k_u (p50 30 $\rightarrow$ 4), and there re-selection wins 1.8 $\times$.

Note that **nation** < 5 constrains the grouping key yet behaves like a unit-side filter, because nation is functionally determined by the customer. The decidable rule is therefore:

Re-run the selection when the filter constrains a grouping-key component that is *not* functionally determined by the privacy unit. Otherwise reuse the frozen value.

4 Privacy validation

Beyond the $\Delta q \leq 1$ construction above, we ran an explicit adversary against the full pipeline, 4×10^6 trials per attack, measuring the advantage $\Pr[\text{say } D' | D'] - \Pr[\text{say } D' | D]$ against the ceiling $(e^\epsilon - 1)/(e^\epsilon + 1)$.

attack	result	verdict
value channel, worst-case target	advantage 0.2587 vs bound 0.2913, <i>scale-free</i>	pass
omniscient attacker (knows all other units)	0.2590 even at a target contributing $4B$	pass
reconstruction of a target's value	posterior sd 1,167 vs prior 1,183 (1.01 $\times$)	pass
reuse of one frozen set over N queries	advantage tracks composed ϵ_v only	pass
adaptive: round 1 picks the round-2 target	<i>loses</i> , 0.36 $\times$ to 0.01 $\times$	pass
SQL join fan-out (182 rows per unit)	$\ \Delta\ _1 = 1.000000 B$ exactly	pass

Two findings came out of this, both about *our own* mechanisms, and both surfaced by explicit neighbour construction after the informal argument had already convinced us.

Finding 1: a clip that was not a norm. Written as $n_u = \sum_g \min(t, B)$ it is a signed sum, which fails open on signed measures. On SUM(shipped - returned) at sf10, **393,053 of 999,982 units exceeded the bound**, worst 2.6 $\times$ — making the release 2.6 ϵ -DP. Fixed by $n_u := \sum_g |\text{clip}(t, -B, B)|$, bit-identical on non-negative data.

Finding 2: δ was under-charged, and an adversary confirms it. A unit creates a group by having a *value* there, not a vote, so with votes truncated to C_e the $k_u - C_e$ unvoted groups each get an independent chance to clear τ on noise alone — chances τ 's union bound never covered:

C_e	k_u	leak, no gate	leak, gated	over by
1	30	2.10×10^{-5}	1.00×10^{-6}	21.0$\times$
1	72	4.70×10^{-5}	1.25×10^{-6}	47.0$\times$
5	72	1.27×10^{-5}	1.75×10^{-6}	12.8$\times$

The fix is one line — release a group only if it has at least one vote — and it is free: on real data it removes 19 of 2,095 groups, each with a 2×10^{-7} chance of release.

Refreshes must be budgeted. R re-releases of frozen metadata give R independent chances: $\Pr[\text{exposed}] = 1 - (1 - p)^R \leq R \delta_0$, confirmed exactly at $R = 1.50$. Batching updates does not reduce the cost of a single refresh; it reduces R .

5 Experimental practice

Several headline figures in this project were produced and later invalidated ($880\times \rightarrow 12\times \rightarrow 4.8\times \rightarrow 4.65\times \rightarrow 1.36\times$ for the clip, and separately $4.5\times \rightarrow 1.9\times$ for the stack), in every case because a baseline had been left at a default while our own arm was tuned. That is why every number here is reported against a baseline tuned over its own free parameters, and why suppressed groups are charged full error. Three further practices were needed, each learned from a result that turned out to be an artifact:

- **Verify sensitivity by neighbour simulation, never by argument.** Both findings in §4 survived a convincing derivation and died on explicit construction.
- **At δ scale, compute exactly rather than simulate.** A Monte Carlo run suggested a $1.4\times$ violation that was pure noise; resolving that at $\delta = 10^{-6}$ needs $\sim 10^8$ trials.
- **Check the grid before believing a negative.** An “oracle bounds” comparison on a $2\times$ -spaced grid — coarser than the $1.3\times$ error basin — reported our own arm getting worse under perfect bounds. Pure artifact.

6 Other ideas we tried

Four other directions were evaluated at length. None is recommended as a headline contribution, for different reasons.

ℓ_1 per-unit norm clip. Bound a unit’s *total* across groups rather than each cell, giving sensitivity B independent of how many groups it touches. Against the published baseline it is worth $3.3\text{--}3.6\times$ within its scope, $1.23\times$ in median across all queries; against a baseline hardened with unpublished improvements it is a wash ($1.01\times$). The scope condition is measurable in advance from the k_u histogram: it needs units touching $\gtrsim 10$ groups, and it *loses* by up to $1.27\times$ when k_u is bimodal, because a few wide-spreading units set B for everybody. Its real argument is not utility but parameter elimination — it has no C_v to select at all.

Frozen partition set. Release the group set once and reuse it, freeing ϵ_η for the values. This works well ($1.2\text{--}4.1\times$, up to $8.2\times$ where τ binds hard) and helps *any* mechanism, ours or the baseline’s. It is however prior art: Tumult Analytics ships `get_groups` + `KeySet`, and Privacy on Beam documents deriving partitions via `SelectPartitions` and passing them as public partitions. What remains unclaimed is narrow — a *persistent* catalogue reused across separately-submitted queries rather than within one pipeline. One caveat found late: freezing *hurts* on unfiltered queries, where the frozen set is the query’s own set and the amortised cost buys nothing.

Gaussian partition selection. Votes have $\ell_1 = k_u$ but $\ell_2 = \sqrt{k_u}$, so Gaussian noise avoids truncation entirely and converts queries that return nothing into ones returning 94% of their key set. Comprehensively pre-empted: implemented in Google’s C++ (`GaussianPartitionSelection`), OpenDP, PipelineDP and Qrlew, and Desfontaines et al. (PoPETs 2022) publish the crossover at $\kappa = 3$ with ϵ/δ -invariance — we independently measured $k_u \approx 4.3$ with the same invariance, which is a rediscovery.

All-or-Frozen thresholding. Replace K per-group release tests with one global AND, saving $\approx (C_u/\epsilon_B) \log C_u$ on the threshold. The privacy analysis is correct — we reproduced the predicted threshold gap to the digit — but the conjunction essentially never fires.

The decisive test is not about the proposal’s design choices. At matched C_u we substituted *Google’s own* statistic (a noised distinct-unit count) into the conjunction: per-group release succeeds for 98.8% of groups under either statistic, and the AND is 0.0% under *both*. The conjunction fails on its own terms, independently of which statistic feeds it and of C_u . The cause is the minimum: an AND over thousands of groups is decided by the smallest, and five of seven tested queries contain a group with a single privacy unit — which no valid threshold can pass, since $\Pr[1 + \text{Lap} \geq \tau] \leq \delta$ forces $\tau \gg 1$. Single-unit groups are precisely why partition selection exists.

A correction we owe the record. We previously reported that reusing the bound-selection histogram, rather than building a dedicated count, costs $736\times$ on the threshold, and drew from it the moral that “a free statistic is not free when it is the wrong statistic”. That comparison was confounded: it changed the budget ($8\times$), the sensitivity $C_u=72 \rightarrow C_e=1$ ($72\times$) and the statistic all at once. Like-for-like the statistic is worth $1.28\times$, and at matched C_u the proposal’s threshold is *below* Google’s ($0.99\text{--}1.28\times$ across settings) because the AND avoids dividing δ by C_u . The reuse was never the defect. What survives is narrower: blocking the conjunction by a unit-determined key component is a valid way to shrink any per-query AND, and the max-bin statistic is genuinely weaker than a count on fine groupings, where it runs at $\approx 0.32\times$ the group’s support.

7 Related work

The primitive is standard. Steps 3–4 are an exponential-mechanism quantile [6]. Gillenwater et al. [4] state exactly this rank-distance utility and prove its L_1 sensitivity is 1 (their Lemma 7), and attribute the single-quantile algorithm to Smith [8], noting it “has since been incorporated into the SmartNoise and IBM differential privacy libraries”. It ships in SmartNoise and IBM diffprivlib. **We claim nothing here**, and in particular the observation that the exponential mechanism needs only *score* sensitivity rather than the sensitivity of the underlying statistic is the defining property of that mechanism, not an insight.

The principle is published. Amin et al. [1] prove that the optimal contribution cap *is* a quantile of the per-user contribution distribution and state explicitly that it must itself be computed privately, citing the DP quantile literature. They leave instantiating it as an open question. Their x_i is records per user — a volume bound.

What is selected elsewhere is a different parameter. The literature contains three distinct per-user bounds, all called “contribution bounds”, and the distinction is not cosmetic: for SUM queries a user may have $\ell_0 = 3$ and $\ell_1 = 10^6$.

bound	meaning	Google’s name
ℓ_0	how many distinct groups one user may touch	<code>max_partitions_contributed</code> (C_u)
ℓ_∞	within one group	<code>max_contributions_per_partition</code>
ℓ_1	one user’s total across all groups	—

The two closest works select the others. [10] (§4.4–4.5) uses the *identical* EM quantile with the *identical* utility function, and even freezes the result with a sparse-vector refresh — but its Theorem 4.2 sums r_i^2/σ_i^2 over all n days with no cap on how many days a user spans, and its Definition 4.1 is “Bounded *Per-day* Contributions”: r_i is an ℓ_∞ bound, and that mechanism has no ℓ_0 bound at all. Liu et al. [5] do use the exponential mechanism to select a bound that constrains ℓ_0 — but as a by-product of ℓ_1 item-sampling (`rand-clip` draws $\min(\|N\|_1, C)$ items, so “each

$\|h_i\|_r \leq C$ for $r = 0, 1, 2, \infty$), only for their unbounded-domain sparse-vector algorithm, and in a COUNT setting where $\ell_0 = \ell_1$ numerically. Their score is expected release error at sensitivity $5C_m/2$, not a rank distance at sensitivity 1. Google’s DP-SQLP [11] recommends the “99th percentile of per-user records... chosen in a DP way” — records again.

The nearest analogue in learning is adaptive clipping [2], which clips to a privately estimated quantile of the gradient-norm distribution, tracked online rather than by exponential mechanism. Same idea one parameter over: an ℓ_2 magnitude bound, not a span bound.

Two works already select ℓ_0 privately. PipelineDP ships an experimental `PrivateLOCalculator` [7] — “*Calculates differentially-private ℓ_0 bound (i.e. `max_partitions_contributed`)*” — applying an exponential mechanism over candidate bounds, scored by a bias/variance objective $-\frac{1}{2}P\sigma - \frac{1}{2}\sum_u \max(\min(k_u, B) - k, 0)$ rather than by rank, and restricted by its own docstring to “*COUNT and PRIVACY_ID_COUNT aggregations*”.

More directly, Cheu et al. [3] (Google, October 2025) write:

“Another approach is to apply a quantile-finding algorithm (Durfee, 2023). For example, **`max_groups_contributed` can be set to, say, the 83rd percentile of the number of groups that a DP unit contributes to.**”

That is private selection of the cross-group span bound as a quantile of the per-unit distinct-group-count distribution, under a `GROUP BY` surface syntax. Their motivation is also ours: “*Previous iterations of federated analytics placed the responsibility of computing such queries on the data analyst. This a point of friction for on-boarding.*” **The mechanism is not new, and this document does not claim it.**

What is left. Three things, none of them the mechanism.

- **Calibration.** They fix $k = 83$ for *sample-complexity* reasons — it minimises their tuning sample size — not for utility. Our sweep (§3.2) finds the utility optimum at $p = 0.95$ – 0.99 , that $p = 0.7$ costs $17.75\times$ worst-case, and that the right p depends on whether C_u is shared with the partition-selection threshold. Their 83rd percentile sits between our best and worst settings, and nothing published says which side to err on.
- **Accounting.** They buy privacy by *splitting the population* — their guarantee rests on “no DP unit is ingested by both autotuning and aggregation” — which discards a sub-population. We pay by composition at $\approx 1\%$ of ϵ . Which is preferable is an open empirical question.
- **Setting.** Theirs is one flat `GROUP BY` over pre-aggregated device uploads; ours is a relational engine where the per-unit group count is derived through joins. The freeze-versus-re-select rule (§3.4) and the inversion under smooth sensitivity are not addressed anywhere we could find.

8 Future work

1. **Compiler-derived public candidate domains.** The proof requires a public $K \geq \max_u k_u$; using the observed maximum makes the output support data-dependent. A SQL compiler should derive a valid domain from grouping-column types, `CHECK` constraints, foreign keys, functional dependencies, public partition catalogues and bounded date ranges. The compiler must also expose when it cannot prove a useful bound. This is necessary for genuinely automatic selection: replacing one analyst-supplied C_u with an equally opaque K is not enough.

2. **Separate existence and value bounds.** Partition release and value release pay for different objects. A bound C_e controls how many groups one unit may name and therefore the partition threshold τ ; a possibly different C_v controls truncation bias and value noise. Selecting (C_e, C_v) jointly may dominate the single `max_partitions_contributed` used by current interfaces. The open technical problem is a private joint score with a useful sensitivity bound, not the non-private grid search used in this report.
3. **Loss-aware selection for general aggregates.** PipelineDP’s private ℓ_0 calculator is documented for COUNT and privacy-ID count. An automatic SQL system must handle SUM, AVG and multiple aggregates sharing one partition-selection channel. The score should predict downstream query loss, including clipping bias and aggregate noise, while remaining private; simply evaluating true query error on each candidate is an oracle, not a deployable selector. The smooth-sensitivity path needs its own objective: the preliminary port in this report shows that the Laplace calibration can invert there.
4. **Joint bound and budget optimisation.** The deployed choice is not C_u alone but $(C_e, C_v, \epsilon_b, \epsilon_\eta, \epsilon_v)$. Our experiments tune these parameters with access to the private truth. Future work should replace that oracle with one accounted private selection step and determine whether the extra choice complexity consumes more privacy than it saves in query error.
5. **A workload-level bound catalogue.** A selected bound should carry its privacy-unit lineage, grouping lineage, predicate assumptions and privacy provenance. Static analysis can reuse it only when those obligations still hold; a filter on a privacy-unit attribute does not in general preserve the k_u distribution. Where invariance cannot be proved, a sparse-vector-style drift monitor could refresh metadata only after a material change. Scheduled refresh remains the baseline: R releases cost up to $R\delta_0$, whether or not the data changed.
6. **Tail-robust private selection.** A dense public domain can contain thousands of candidates above the observed maximum. When those candidates have equal or nearly equal rank score, their combined probability can dominate the exponential mechanism even though each is individually poor. Candidate lattices, base measures or alternative ordered-selection mechanisms should be evaluated with a requirement that widening a valid public domain does not radically inflate the selected bound.
7. **Privacy-certified compilation.** An EXPLAIN PRIVACY mode should report the adjacency relation, derived candidate domain, selected or reused (C_e, C_v) , sensitivity of every release, partition-selection threshold, metadata provenance and the complete budget split. This turns automatic selection into an auditable compiler decision rather than hidden preprocessing.

Preliminary mechanism-only probe. We added `attacks/automatic_cu_future.py` to test the non-compiler parts on privacy-ID COUNT over the real TPC-H `month|nation` incidence relation. The selector uses the public group-domain bound $K = 2095$, not the observed maximum; utility configurations are chosen on four simulation draws and reported on twelve independent draws. “Loss oracle” therefore measures headroom, not a deployable private selector:

units	observed max	original: $\Pr[\hat{C} > \max k_u]$	direct+hybrid	shared oracle	separate oracle
181,532	72	97.4%	4.5%	5.52%	3.82%
45,320	66	99.2%	69.2%	78.48%	41.56%

Removing the noisy- N margin and replacing the dense upper tail with a fixed hybrid lattice solve the large-population failure but not the smaller one: at $p = 0.99$ and 0.008 selection budget, candidate multiplicity still overwhelms the rank gap. Separate bounds show real headroom in

both cases ($1.45\times$ and $1.89\times$ over the shared loss oracle), which justifies pursuing a private joint score; the high absolute error in the smaller population also shows that automatic parameter selection cannot repair a partition-selection channel with inadequate support.

All seven directions are system or calibration problems, not claims that the underlying exponential-mechanism quantile is new. The first integrated implementation should include the two safety modifications from §4 and must be evaluated through compiled SQL rather than simulation alone.

References

- [1] Kareem Amin, Alex Kulesza, Andrés Muñoz Medina, and Sergei Vassilvitskii. Bounding user contributions: A bias-variance trade-off in differential privacy. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 263–271. PMLR, 2019.
- [2] Galen Andrew, Om Thakkar, H. Brendan McMahan, and Swaroop Ramaswamy. Differentially private learning with adaptive clipping. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [3] Albert Cheu, Artem Lagzdin, Brett McLarnon, Daniel Ramage, Katharine Daly, Marco Gruteser, Peter Kairouz, Rakshita Tandon, Stanislav Chiknavaryan, Timon Van Overveldt, and Zoe Gong. Toward provably private analytics and insights into GenAI use. *arXiv preprint arXiv:2510.21684*, 2025.
- [4] Jennifer Gillenwater, Matthew Joseph, and Alex Kulesza. Differentially private quantiles. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 3713–3722. PMLR, 2021.
- [5] Yuhan Liu, Ananda Theertha Suresh, Wennan Zhu, Peter Kairouz, and Marco Gruteser. Algorithms for bounding contribution for histogram estimation under user-level privacy. In *International Conference on Machine Learning (ICML)*, 2023.
- [6] Frank McSherry and Kunal Talwar. Mechanism design via differential privacy. In *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 94–103, 2007.
- [7] PipelineDP contributors. PrivateL0Calculator: differentially private selection of `max_partitions_contributed`. <https://github.com/OpenMined/PipelineDP>, `pipeline_dp/private_contribution_bounds.py`, 2023.
- [8] Adam Smith. Privacy-preserving statistical estimation with optimal convergence rates. In *Proceedings of the 43rd Annual ACM Symposium on Theory of Computing (STOC)*, pages 813–822, 2011.
- [9] Royce J. Wilson, Celia Yuxin Zhang, William Lam, Damien Desfontaines, Daniel Simmons-Marengo, and Bryant Gipson. Differentially private SQL with bounded user contribution. *Proceedings on Privacy Enhancing Technologies*, 2020(2):230–250, 2020.
- [10] Yingtai Xiao, Jian Du, Shikun Zhang, Wanrong Zhang, Qiang Yan, Danfeng Zhang, and Daniel Kifer. Click without compromise: Online advertising measurement via per user differential privacy. *arXiv preprint arXiv:2406.02463*, 2024.
- [11] Bing Zhang, Vadym Doroshenko, Peter Kairouz, Thomas Steinke, Abhradeep Thakurta, Ziyin Ma, Eidan Cohen, Himani Apte, and Jodi Spacek. Differentially private stream processing at scale. *arXiv preprint arXiv:2303.18086*, 2023.